

TYPESCRIPT

Interview Cheatsheet — Clean 2–3 Line Definitions

Type System · Generics · Interfaces · Utility Types · Advanced Types

What's inside

1. Core Type System
2. Interfaces & Type Aliases
3. Functions & Classes
4. Generics
5. Union, Intersection & Narrowing
6. Advanced Types
7. Utility Types
8. Decorators
9. Modules & Declaration Files
10. Compiler & Tooling

1. Core Type System

Static Typing

TypeScript checks types at compile time rather than runtime, catching type mismatches before code ever executes — it compiles down to plain JavaScript.

Type Inference

TypeScript automatically determines a variable's type from its initial value without an explicit annotation, e.g. `let x = 5` infers `number`.

any vs unknown

`any` disables type checking entirely for a value; `unknown` is type-safe — it accepts any value but requires a type check/narrowing before you can use it.

never

Represents a value that never occurs — used as the return type of functions that always throw or infinite-loop, and to catch impossible cases in exhaustiveness checks.

void

Represents the absence of a return value, typically used as the return type of functions that don't return anything meaningful (as opposed to `undefined` explicitly).

Type Annotation vs Type Assertion

An annotation (`let x: number`) declares a type for the compiler to enforce; an assertion (`x as number`) overrides the compiler's inferred type without runtime conversion.

Structural Typing

TypeScript compares types based on their shape (which properties/methods exist) rather than their declared name — two unrelated types are compatible if their structures match.

Strict Mode (strict: true)

A compiler flag bundling stricter checks like `strictNullChecks` and `noImplicitAny`, catching more potential bugs at the cost of needing more precise typing.

2. Interfaces & Type Aliases

Interface

Declares the shape of an object — its properties, methods, and their types — used primarily to define contracts for objects and classes.

Type Alias

Creates a name for any type (object shapes, unions, primitives, tuples) using `type`. More flexible than interfaces since it can alias non-object types too.

Interface vs Type Alias

Interfaces support declaration merging and are generally preferred for object shapes/class contracts; type aliases are needed for unions, tuples, and mapped types.

Declaration Merging

When multiple interfaces with the same name are declared, TypeScript automatically merges their members into one combined interface.

Optional & Readonly Properties

`prop?: type` marks a property as optional; `readonly prop: type` prevents reassignment after the object's initial creation.

Index Signatures

Allows an object type to have an unknown number of properties of a consistent type, e.g. `{ [key: string]: number }`, useful for dictionary-like objects.

extends (Interface Inheritance)

Lets one interface inherit the members of another (or multiple), building up composite shapes without duplicating property definitions.

3. Functions & Classes

Function Type Annotations

Explicitly types a function's parameters and return value, e.g. `(a: number, b: number) => number`, ensuring callers pass and receive the correct types.

Optional & Default Parameters

`param?: type` makes a parameter optional (implicitly `| undefined`); default parameters (`param: type = value`) supply a fallback if omitted.

Function Overloads

Declaring multiple call signatures for the same function name so TypeScript can validate different parameter/return combinations for the same implementation.

Access Modifiers

`public` (default) is accessible anywhere; `private` only within the declaring class; `protected` within the class and its subclasses.

Abstract Classes

Classes marked `abstract` that cannot be instantiated directly and may declare abstract methods that subclasses are required to implement.

Parameter Properties

A shorthand where constructor parameters prefixed with an access modifier (e.g. `private name: string`) are automatically declared and assigned as class properties.

implements keyword

Used on a class to enforce that it satisfies a given interface's shape, giving compile-time verification that all required members are implemented.

4. Generics

Generics

Allow types, interfaces, and functions to be written once and reused with different types, preserving type safety instead of falling back to `any`, e.g. `Array<T>`.

Generic Constraints

Restrict a generic type parameter to types matching a given shape using `extends`, e.g. `<T extends {length: number}>`, so only compatible types are accepted.

Generic Defaults

Provide a fallback type for a generic parameter if the caller doesn't specify one, e.g. `interface Box<T = string>`.

keyof with Generics

Combining `keyof T` with a generic parameter lets a function safely restrict an argument to only the actual property names of a given type.

Generic Classes

Classes parameterized with a type variable, e.g. `class Stack<T>`, so the same implementation works safely across different data types.

5. Union, Intersection & Narrowing

Union Type

A type that can be one of several types, written with `|`, e.g. `string | number` — a value must match at least one of the listed types.

Intersection Type

Combines multiple types into one that must satisfy all of them simultaneously, written with `&`, merging their members together.

Type Narrowing

The process of refining a broader type (like a union) to a more specific one within a code branch, using checks like `typeof`, `instanceof`, or truthiness.

Discriminated Unions

A union of object types sharing a common literal "tag" property, letting TypeScript narrow to the exact variant based on that tag's value in a switch/if.

Type Guards

Functions returning a special `arg is Type` predicate that TypeScript uses to narrow a variable's type within the block where the guard returns true.

Literal Types

Types that represent exact values rather than a general primitive, e.g. `"success" | "error"`, often combined with unions to model finite states.

Non-null Assertion (!)

The `!` operator tells the compiler to treat a value as non-null/non-undefined even though its type includes those, without any runtime check.

6. Advanced Types

Mapped Types

Types that build new object types by transforming each property of an existing type, e.g. `{ [K in keyof T]: T[K] }`, the mechanism behind utility types like `Partial`.

Conditional Types

Types that choose between two types based on a condition, written as `T extends U ? X : Y`, enabling type-level branching logic.

infer keyword

Used inside a conditional type to extract and capture a sub-type for reuse, e.g. pulling out a function's return type or a Promise's resolved value.

Template Literal Types

Types built using template-literal-like syntax on string literal types, e.g. ``${on}${Capitalize<T>}``, useful for generating event-name or key patterns.

Tuple Types

Arrays with a fixed number of elements where each position has a known, potentially different type, e.g. `[string, number]`.

Type Guards via typeof/instanceof

Built-in runtime checks TypeScript understands for narrowing: `typeof` for primitives, `instanceof` for class instances.

7. Utility Types

Partial<T> / Required<T>

`Partial` makes all properties of `T` optional; `Required` makes all properties mandatory, removing any optional modifiers.

Pick<T, K> / Omit<T, K>

`Pick` constructs a type with only the selected keys `K` from `T`; `Omit` constructs a type with all of `T`'s keys except those in `K`.

Readonly<T>

Constructs a type where all properties of `T` are marked `readonly`, preventing reassignment after object creation.

Record<K, T>

Constructs an object type with keys of type `K`, each mapped to a value of type `T` — commonly used to type dictionaries and lookup maps.

Exclude<T, U> / Extract<T, U>

`Exclude` removes members of union `T` that are assignable to `U`; `Extract` keeps only members of `T` that are assignable to `U`.

ReturnType<T> / Parameters<T>

`ReturnType` extracts a function type's return type; `Parameters` extracts a tuple of its parameter types — both rely on `infer` internally.

NonNullable<T>

Removes `null` and `undefined` from a type `T`, useful after narrowing to guarantee a value is definitely present.

8. Decorators

Decorator

A special declaration (`@decoratorName`) attached to a class, method, property, or parameter that can observe, modify, or replace its behavior at definition time.

Class Decorator

Applied to a class declaration, receiving the constructor function — can be used to log, seal, or augment the class, as seen in Angular's `@Component`.

Method & Property Decorators

Applied to individual methods or properties to intercept access, add validation, or wrap behavior — the basis for many framework features like memoization or logging.

Decorator Factory

A function that returns a decorator, allowing configuration to be passed in, e.g. `@Component({selector: 'app-root'})` where the outer call configures the decorator.

9. Modules & Declaration Files

Type Declaration Files (.d.ts)

Files containing only type information (no implementation), used to describe the shape of existing JavaScript libraries so TypeScript can type-check their usage.

DefinitelyTyped (@types)

A community repository of type declaration packages (installed as `@types/package-name`) providing types for JS libraries that don't ship their own.

Ambient Declarations

Declarations made with the `declare` keyword that describe types/values existing elsewhere (e.g. globals, external scripts) without providing an implementation.

Module Augmentation

Extending an existing module's or global type's shape from a different file, commonly used to add custom properties to third-party library types.

Namespace

An older TypeScript-specific way to organize code into logical groups under a shared name, largely superseded by ES modules in modern codebases.

10. Compiler & Tooling

tsconfig.json

The configuration file controlling how the TypeScript compiler behaves — target JS version, module system, strictness flags, included/excluded files, and more.

target & module

`target` sets which JS version the compiler outputs (e.g. ES2020); `module` sets the module system used in the output (CommonJS, ESNext, etc.).

strictNullChecks

A compiler flag that makes `null` and `undefined` distinct types not assignable to other types unless explicitly included, catching a large class of null-reference bugs.

Type Checking vs Transpilation

TypeScript's compiler both checks types for errors and transpiles TS syntax down to plain JS; tools like Babel/esbuild can transpile without type-checking for speed.

Path Aliases

Configured in `tsconfig.json` via `paths / baseUrl`, letting imports use short aliases (e.g. `@app/utils`) instead of long relative paths.

Incremental Compilation

The compiler caches build information (`.tsbuildinfo`) between builds so subsequent compiles only re-check changed files, speeding up large project builds.

Prepared as a quick-revision reference for TypeScript interview prep — covers the type system fundamentals through advanced generics and compiler internals.